

ANALIZA I PRZETWARZANIE OBRAZÓW

Football tracker

Detekcja i śledzenie zawodników na nagraniach meczów piłkarskich

Sprawozdanie z projektu

Autorzy:

Bartłomiej Obrochta
Kamil Milewski
Maciej Leśniak

17 czerwca 2026

Spis treści

1	Temat i założenia projektu	2
2	Zbiory danych	2
2.1	Zbiory do detekcji zawodników (4 klasy)	2
2.2	Zbiory do detekcji punktów boiska (1 klasa + 32 keypointy)	3
2.3	Łączenie i wstępne przetwarzanie zbioru boiska	3
2.4	Augmentacja offline modelu boiska	4
2.5	Opcje skryptu <code>dataset.py</code>	4
3	Struktura projektu	5
4	Konfiguracja i przebieg uczenia	5
4.1	Trenowane modele	5
4.2	Skrypty i konfiguracje treningu	5
4.3	Dwuetapowe trenowanie modelu boiska	6
4.4	Generowanie minimapy 2D i wyznaczanie pozycji zawodników	6
5	Wyniki i działanie programu	7
5.1	Co program robi po uruchomieniu	7
5.2	Wyniki ilościowe (<code>process.py</code>)	9
5.3	Stabilność homografii (<code>eval.py</code> <code>homography</code>)	10
5.4	Co działa, a co nie	11
6	Poradnik uruchomieniowy	11
6.1	Instalacja (conda + CUDA)	11
6.2	Najważniejsza uwaga: kodek wideo	12
6.3	Skrypty uruchomieniowe	12
6.4	Najszybsza weryfikacja działania	12
7	Podsumowanie	13

1 Temat i założenia projektu

Celem projektu **Football tracker** jest stworzenie kompletnego potoku przetwarzania obrazu, który na podstawie nagrania meczu piłkarskiego automatycznie wykrywa zawodników (w tym osobno bramkarzy), sędziów i piłkę, śledzi ich pomiędzy kolejnymi klatkami, a następnie rzutuje ich pozycje na schematyczny widok boiska z lotu ptaka (minimapę 2D) i wylicza proste statystyki, takie jak pokonany dystans i prędkość poszczególnych zawodników.

Projekt obejmuje:

- detekcję zawodników, piłki i sędziów,
- trwałe śledzenie tożsamości (ID) obiektów pomiędzy klatkami, czyli wyznaczanie trajektorii,
- odwzorowanie ramek otaczających (*bounding boxów*) na widok boiska z góry,
- wyznaczanie homografii *per klatka* na podstawie wyuczonych punktów charakterystycznych boiska,
- obliczanie pokonanego dystansu i prędkości każdego zawodnika,
- potok wsadowy (`process.py`) generujący gotowy plik wideo wraz z tabelą podsumowującą oraz dwuetapowy schemat trenowania modelu boiska.

Architektura potoku jest dwumodelowa i przebiega następująco:

YOLO26 (detektor) → ByteTrack (tracking) →
YOLO26-pose (32 keypointy boiska) → homografia →
minimapa 2D + dystans/prędkość.

Dwoma trenowanymi komponentami są: **detektor obiektów** (zawodnik, bramkarz, sędzia, piłka) oraz **model punktów charakterystycznych boiska** (model typu *pose* z 32 punktami). Komponenty śledzące (ByteTrack) oraz moduł homografii nie są uczone – są to klasyczne algorytmy (filtr Kalmana + algorytm węgierski w trackerze, RANSAC w homografii).

2 Zbiory danych

Projekt korzysta z czterech zbiorów danych. Dwa z nich służą do uczenia detektora zawodników, a dwa – do uczenia modelu punktów boiska. Co istotne, katalogi z danymi pobiera się skryptem `dataset.py`.

2.1 Zbiory do detekcji zawodników (4 klasy)

Schemat klas jest wspólny dla obu źródeł detekcyjnych i zdefiniowany w pliku `configs/classes.yaml`:

ID	Klasa
0	player (zawodnik z pola)
1	goalkeeper (bramkarz)
2	referee (sędzia)
3	ball (piłka)

Roboflow Players – zbiór `football-players-detection-3zvbc` (wersja 12) z platformy Roboflow Universe. Zawiera pojedyncze klatki z nagrań meczowych, na których obiekty (zawodnicy, bramkarze, sędziowie, piłka) są opisane prostokątnymi ramkami otaczającymi (ok. 600 klatek). Zbiór jest udostępniany od razu w formacie YOLO; pobierany jest w formacie `yolo8`. Etykiety

oryginalne mają inną kolejność klas, dlatego po pobraniu są przemapowywane do schematu kanonicznego (`ball`→3, `goalkeeper`→1, `player`→0, `referee`→2).

SoccerNet Tracking – opcjonalny, większy zbiór z projektu SoccerNet (dostęp po zaakceptowaniu NDA i podaniu hasła). Zawiera sekwencje klatek z meczów z adnotacjami w stylu MOT (`gt.txt`: numer klatki, ID obiektu, pozycja ramki, klasa). W projekcie dane są konwertowane do formatu YOLO, a klasy przemapowywane (1→`player`, 2→`goalkeeper`, 3→`referee`, 4→`ball`). Heurystyka podziału: klipy ze słowem `test` w ścieżce trafiają do zbioru walidacyjnego, pozostałe do treningowego.

Oba zbiory detekcyjne łączone są poleceniem `python dataset.py merge` do katalogu `data/processed/combined` (z prefiksami `soccernet__` oraz `roboflow__`), co generuje plik `configs/combined.yaml` używany w treningu i ewaluacji detektora.

2.2 Zbiory do detekcji punktów boiska (1 klasa + 32 keypointy)

Model boiska to model typu *pose*: wykrywa jedną ramkę klasy `pitch` (cały widoczny obszar boiska) oraz 32 punkty charakterystyczne. Kanoniczny schemat 32 punktów (w metrach, dla boiska FIFA 105 × 68 m) jest zdefiniowany w `configs/pitch_keypoints.yaml` i obejmuje m.in.: cztery rogi boiska, końce linii środkowej, narożniki pól karnych i bramkowych, punkty przy słupkach, punkt środkowy, punkty karne oraz wierzchołek łuku pola karnego.

Roboflow Pitch – zbiór `football-field-detection-f07vi` (wersja 15). Zawiera obrazy boiska, najczęściej w szerokich ujęciach taktycznych, z naniesionymi 32 punktami charakterystycznymi w formacie YOLO-pose. Według dokumentacji ok. 317 klatek. Pobierany jest w formacie `yolo26`, a do `data.yaml` wstrzykiwane są pola `kpt_shape`: [32, 3] oraz `flip_idx` (mapa odbicia lustrzanego punktów lewa/prawa strona boiska).

SoccerNet Calibration 2023 – zbiór do kalibracji geometrii boiska (ok. 13 tys. klatek broadcastowych). Nie zawiera ramek zawodników; zawiera adnotacje *linii* boiska. Projekt przelicza te linie na 32 punkty – przez wyznaczanie przecięć par linii (np. linia boczna ∩ linia bramkowa = róg boiska). Punkty 28–31 (punkt środkowy, punkty karne, wierzchołek łuku) nie występują w adnotacjach SoccerNet i pozostają niewidoczne dla tego źródła.

2.3 Łączenie i wstępne przetwarzanie zbioru boiska

Po pobraniu obu źródeł boiska są one łączone i poddawane *green suppression* – tłumieniu kanału zielonego, dzięki czemu białe linie boiska stają się bardziej wyraziste względem trawy. Tę samą operację stosuje się identycznie podczas uczenia i podczas wnioskowania, co zapewnia spójność cech. Jej implementacja jest bardzo zwięzła (Listing 1).

```
1 def enhance_pitch_lines(frame: np.ndarray, green_factor: float = 0.75) -> np.ndarray:
2     """Suppress the green channel to make white pitch lines stand out.
3     Grass pixels have G >> R,B. White lines have R~G~B (high brightness).
4     Multiplying the G channel by < 1 darkens grass without affecting lines."""
5     out = frame.astype(np.float32)
6     out[:, :, 1] *= green_factor # suppress green
7     return np.clip(out, 0, 255).astype(np.uint8)
```

Listing 1: Tłumienie kanału zielonego – `src/football_tracker/pitch/preprocess.py`

Wynikowy zbiór (`data/processed/combined_pitch_gs/`) liczy ok. 13,3 tys. obrazów treningowych i ok. 2,5 tys. walidacyjnych, przy czym zachowywane są tylko etykiety z co najmniej 4 widocznymi punktami (4 to geometryczne minimum do wyznaczenia homografii).

2.4 Augmentacja offline modelu boiska

Wbudowana augmentacja YOLO dobrze radzi sobie z geometrią, ale słabo z różnorodnością fotometryczną oraz z kadratami pokazującymi tylko część boiska. Ponieważ wszystkie klatki źródłowe pokazują całe boisko, model uczy się np., że „linia środkowa $\approx 50\%$ szerokości obrazu”, a następnie myli punkty przy zbliżeniach na jedną połowę. Rozwiązuje to augmentacja *offline* (moduł `augment_pitch_dataset.py`, wymaga biblioteki `albumations`), która dla każdego obrazu źródłowego zapisuje na dysk:

- 1 dowiązanie do oryginału,
- 2 kopie z augmentacją fotometryczną (jasność/kontrast, HSV, gamma, CLAHE, rozmycie, szum, kompresja JPEG, sporadycznie skala szarości, wycinanie fragmentów oraz drobne przekształcenia geometryczne),
- **11 strategicznych wycinków** fragmentów boiska (połówki, ćwiartki, środek, zbliżenia na pola bramkowe) – każdy także augmentowany fotometrycznie.

Daje to do ok. 14 próbek na obraz źródłowy, czyli ok. 110 tys. obrazów treningowych. Próbki, w których po przekształceniu zostaje mniej niż 4 widoczne punkty, są odrzucane. Zbiór walidacyjny domyślnie nie jest augmentowany (aby metryki pozostały porównywalne). Tryb `-partials-only` zapisuje wyłącznie 11 wycinków na obraz (ok. 80 tys. obrazów) i służy do etapu dostrajania.

2.5 Opcje skryptu `dataset.py`

Polecenie	Działanie	Wyjście
<code>soccernet</code>	pobiera SoccerNet (wymaga hasła NDA) i konwertuje do YOLO	<code>data/raw/soccernet</code>
<code>roboflow</code>	pobiera zbiór graczy z Roboflow (wymaga klucza API)	<code>data/raw/roboflow</code>
<code>pitch</code>	pobiera zbiór punktów boiska z Roboflow	<code>data/raw/pitch</code>
<code>soccernet-pitch</code>	konwertuje SoccerNet calibration-2023 do YOLO-pose	<code>data/raw/soccernet_pitch</code>
<code>preprocess-pitch</code>	green suppression + scalenie źródeł boiska	<code>data/processed/combined_pitch_gs</code>
<code>augment-pitch</code>	augmentacja offline (fotometria + wycinki)	<code>data/processed/combined_pitch_gs_aug</code>
<code>merge</code>	scalenie zbiorów detekcji graczy	<code>data/processed/combined</code>
<code>youtube</code>	pobranie klipu z YouTube (yt-dlp)	<code>data/raw/youtube</code>
<code>all</code>	roboflow + pitch + merge (ścieżka bez NDA)	<code>data/processed/combined</code>

Zmienne środowiskowe: `ROBOFLOW_API_KEY` (zbiory Roboflow) oraz `SOCERNET_PASSWORD` (SoccerNet). Przykładowo pełna ścieżka przygotowania danych boiska wygląda tak:

```
export ROBOFLOW_API_KEY=... SOCCERNET_PASSWORD=...
python dataset.py pitch # Roboflow, ~317
    klatek
python dataset.py soccernet-pitch --zip /sciezka/train.zip # SoccerNet, ~13k
    klatek
python dataset.py preprocess-pitch # green suppression +
    scalenie
python dataset.py augment-pitch # augmentacja offline
    (~110k)
```

3 Struktura projektu

Repozytorium jest zorganizowane wokół czterech skryptów wejściowych i biblioteki `football_tracker` w katalogu `src/`.

Ścieżka	Rola
<code>dataset.py</code>	pobieranie, konwersja, scalanie, preprocessing i augmentacja danych
<code>train.py</code>	trenowanie detektora i modelu boiska (jednoetapowo) + eksport ONNX
<code>train_pitch_partials.py</code>	etap 1 treningu boiska: od zera na zbiorze z wycinkami
<code>train_pitch_finetune.py</code>	etap 2 treningu boiska: dostrajanie na samych wycinkach
<code>eval.py</code>	ewaluacja detektora / modelu boiska / stabilności homografii
<code>demo.py</code>	interaktywny podgląd całego potoku w oknie
<code>process.py</code>	wsadowe przetwarzanie wideo: film → film z adnotacjami + raport
<code>Makefile</code>	skrót do typowych przepływów (<code>make install-cuda</code> , <code>make smoke</code> , ...)
<code>configs/</code>	pliki YAML: schematy klas/punktów oraz konfiguracje treningu
<code>samples/clips/</code>	5 przykładowych klipów (fragmenty meczu Liverpool-PSG, 1080p)
<code>doc/</code>	niniejsze sprawozdanie wraz z ilustracjami w <code>doc/img/</code>

W katalogu `src/football_tracker/` wyodrębniono moduły: `datasets/` (pobieranie i przetwarzanie zbiorów), `training/` (trenery, ewaluacja, eksport, budowa raportów), `tracking/` (opakowanie ByteTrack oraz bufor śladów), `pitch/` (schemat punktów, homografia dynamiczna, minimapa, preprocessing), `analytics/` (dystans/prędkość oraz mapa cieplna), `pipeline/live.py` (rdzeń potoku łączący wszystkie etapy) oraz `reporting.py` (zapisywanie raportów JSON). Wy-trenowane modele zapisywane są w `models/checkpoints/`, a raporty z uruchomień w `outputs/reports/`.

4 Konfiguracja i przebieg uczenia

4.1 Trenowane modele

Trenowane są dwa niezależne modele:

1. **Detektor zawodników** – architektura YOLO26-s, 4 klasy (zawodnik, bramkarz, sędzia, piłka). Jego zadaniem jest wskazanie na każdej klatce, *gdzie* znajdują się obiekty gry. Wynik zapisywany jest do `models/checkpoints/best.pt`.
2. **Model punktów boiska** – architektura YOLO26s-pose, jedna klasa (`pitch`) i 32 punkty charakterystyczne. Jego zadaniem jest wskazanie znanych punktów boiska, na podstawie których wyznacza się homografię (przekształcenie obrazu kamery na współrzędne boiska). Wynik zapisywany jest do `models/checkpoints/pitch.pt`.

Oba modele są dostrajane z wag startowych YOLO26 (transfer learning); w modelu boiska głowica punktów jest reinicjalizowana dla 32 punktów (uczona od zera).

4.2 Skrypty i konfiguracje treningu

Trening uruchamia się skryptem `train.py`, który wczytuje plik YAML i przekazuje go bezpośrednio do funkcji `ultralytics.YOLO.train()`:

```
python train.py detector # detektor wg configs/training.yaml -> best.pt
python train.py pitch   # boisko   wg configs/training_pitch.yaml -> pitch.pt
python train.py all      # oba modele po kolei
```

```
python train.py export --weights models/checkpoints/best.pt # eksport do ONNX
```

Najważniejsze parametry uczenia zebrano w tabeli poniżej (źródła: `configs/training.yaml` oraz `configs/training_pitch.yaml`).

Parametr	Detektor	Model boiska
Architektura	yolo26s.pt	yolo26s-pose.pt
Zbiór (data)	configs/combined.yaml	combined_pitch_gs/data.yaml
Liczba epok	120	120
Rozdzielczość (imgsz)	1280	960
Batch	16	16
Optymalizator	AdamW	AdamW
lr0	0,0005	0,001
Augmentacja mozaiki	1,0	0,5 (wyłączana na 15 ost. epok)
Specyficzne	mixup 0,1	pose 12,0; kobj 2,0; crop_fraction 0,7
Cierpliwość (patience)	30	25
Wynik	best.pt	pitch.pt

Wartość `lr0=0,001` dla modelu boiska jest celowo wysoka – głowica punktów uczona jest praktycznie od zera, a niższy współczynnik powodował, że model utykał w lokalnym optimum.

4.3 Dwuetapowe trenowanie modelu boiska

Dla lepszej obsługi kadrów częściowych przewidziano wariant dwuetapowy:

- **Etap 1** (`train_pitch_partials.py`, konfiguracja `training_pitch_partials.yaml`) – trening od zera na zbiorze z augmentacją offline (oryginały + fotometria + 11 wycinków), 60 epok, `lr0=0,001`. Wynik nadpisuje `pitch.pt`.
- **Etap 2** (`train_pitch_finetune.py`, konfiguracja `training_pitch_partials_finetune.yaml`) – dostrajanie modelu z etapu 1 wyłącznie na wycinkach, 30 epok, `lr0=0,00005` (20-krotnie niższy, by chronić istniejące wagi). Wynik trafia do `pitch_finetuned.pt` i *nie* nadpisuje `pitch.pt`.

Każde uruchomienie treningu zapisuje checkpoint oraz raport JSON (z krzywymi straty i metrykami najlepszej epoki) w `outputs/reports/train/`.

4.4 Generowanie minimapy 2D i wyznaczanie pozycji zawodników

To kluczowy element łączący oba modele. Dla każdej klatki model boiska zwraca punkty charakterystyczne, z których estymator homografii wyznacza macierz przekształcenia obrazu na współrzędne boiska. Najpierw odrzucane są punkty o niskiej pewności, sprawdzane jest minimum 6 widocznych punktów, a następnie dopasowanie wykonywane jest metodą RANSAC (Listing 2).

```
1 image_pts = kpts[mask, :2].astype(np.float32) # punkty na obrazie (px)
2 world_pts = self.scheme.world_xy[mask].astype(np.float32) # punkty boiska (m)
3
4 H, inlier_mask = cv2.findHomography(
5     image_pts, world_pts, cv2.RANSAC,
6     RANSAC_REPROJ_PX, maxIters=RANSAC_MAX_ITERS,
7 )
8 if H is None or inlier_mask is None:
9     return self._fallback(n_visible, 0, "RANSAC failed")
10 n_inliers = int(inlier_mask.sum())
11 if n_inliers < MIN_INLIERS:
12     return self._fallback(n_visible, n_inliers, f"only {n_inliers} inliers")
```

Listing 2: Rdzeń dopasowania homografii – `pitch/dynamic_homography.py`

Surowe dopasowanie jest dodatkowo chronione zestawem zabezpieczeń: kontrolą rozpiętości punktów (odrzućcie, gdy punkty tworzą zbyt mały klaster), kontrolą orientacji (kolejność rogów w pikselach musi zgadzać się z kolejnością w świecie, co wyłapuje lustrzane odbicia minimapy) oraz odrzuceniem „dzikich” dopasowań skaczących o ponad 40 m względem wygładzonej macierzy. Macierz jest wygładzana wykładniczo (EMA) między klatkami, a w razie chwilowego braku dopasowania utrzymywana jest ostatnia poprawna homografia (po dłuższej przerwie minimapa jest podbarwiana na bursztynowo).

Mając zaufaną homografię, dla każdego śledzonego zawodnika bierze się *punkt stopy* (środek dolnej krawędzi ramki) i rzutuje go na metry boiska; przepuszczane są wyłącznie punkty mieszczące się w granicach boiska (Listing 3). Tak otrzymane współrzędne trafiają na minimapę oraz do modułu liczącego dystans i prędkość.

```

1 if tracks and homo is not None and homo_trusted:
2     foot_pts = np.array(
3         [(t.xyxy[0] + t.xyxy[2]) / 2, t.xyxy[3]] for t in tracks],
4         dtype=np.float32,
5     )
6     pitch_pts = homo.project(foot_pts)           # piksele -> metry boiska
7     in_pitch = (
8         (pitch_pts[:, 0] >= -_MARGIN) & (pitch_pts[:, 0] <= _PITCH_W + _MARGIN) &
9         (pitch_pts[:, 1] >= -_MARGIN) & (pitch_pts[:, 1] <= _PITCH_H + _MARGIN)
10    )

```

Listing 3: Rzutowanie pozycji zawodników na boisko – pipeline/live.py

Prędkość liczona jest tylko na klatkach ze *świeżym* dopasowaniem homografii – dzięki temu ruch kamery podczas ekstrapolacji nie jest mylnie interpretowany jako ruch zawodnika. Stosowane są ponadto: medianowe okno wygładzające, próg szumu (drobne drgania rzutowania = 0 km/h) oraz twardy limit 40 km/h.

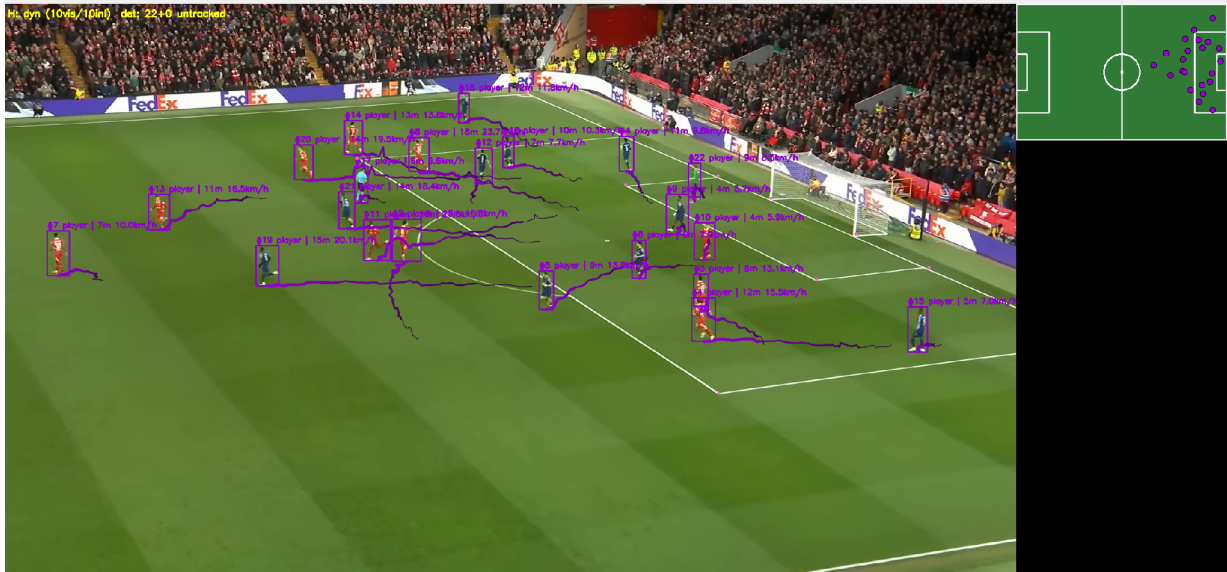
5 Wyniki i działanie programu

5.1 Co program robi po uruchomieniu

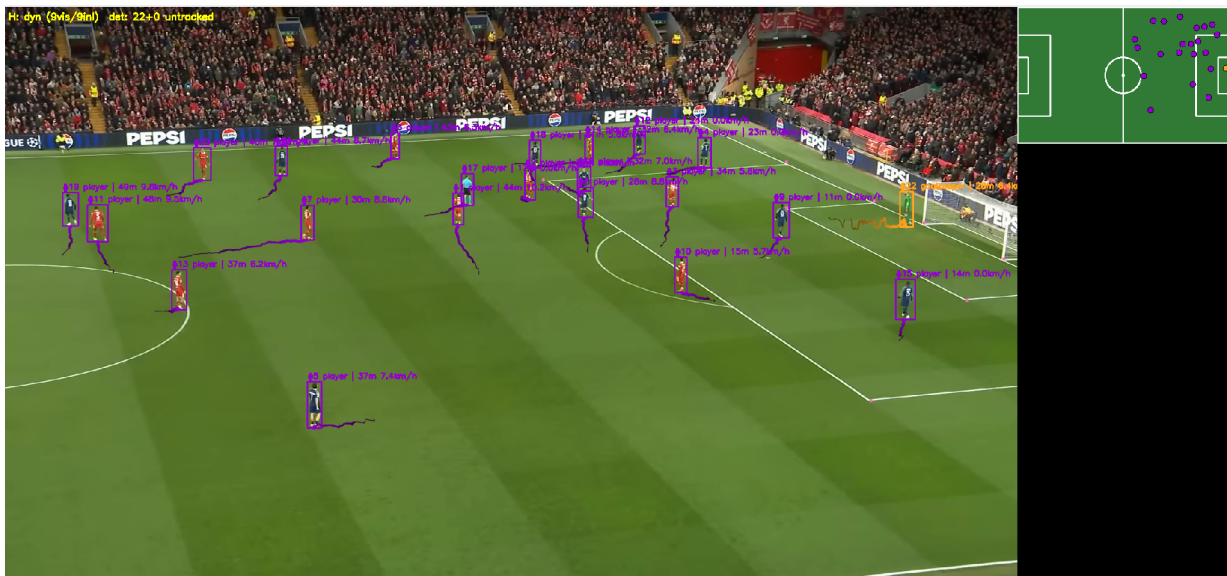
Po uruchomieniu na nagraniu meczu program dla każdej klatki:

- **wykrywa zawodników i bramkarzy** – są one rozpoznawane bardzo niezawodnie i otaczane kolorowymi ramkami;
- **nie zawsze wykrywa piłkę i sędziów** – te klasy pojawiają się znacznie rzadziej (piłka jest mała i szybka, sędziowie często przebywają na obrzeżach kadru);
- **śledzi obiekty** – każdemu nadawany jest trwały numer ID, a za poruszającymi się zawodnikami rysowany jest zanikający ślad (linia trajektorii w stylu „ogona komety”): najnowszy fragment jest jasny i grubszy, najstarszy ciemniejszy i cieńszy;
- **wypisuje etykiety** nad ramką: numer ID, klasę oraz – dla zawodników – pokonany dystans i bieżącą prędkość (np. #7 player | 41m 7.0km/h); prędkość pojawia się dopiero po ustabilizowaniu śledzenia;
- **rysuje punkty boiska** (różowe kropki) oraz pasek statusu homografii w lewym górnym rogu (np. H: dyn (10vis/10in1));
- **generuje minimapę 2D** po prawej stronie kadru, na której pozycje zawodników są naniesione jako punkty na schemacie boiska z lotu ptaka.

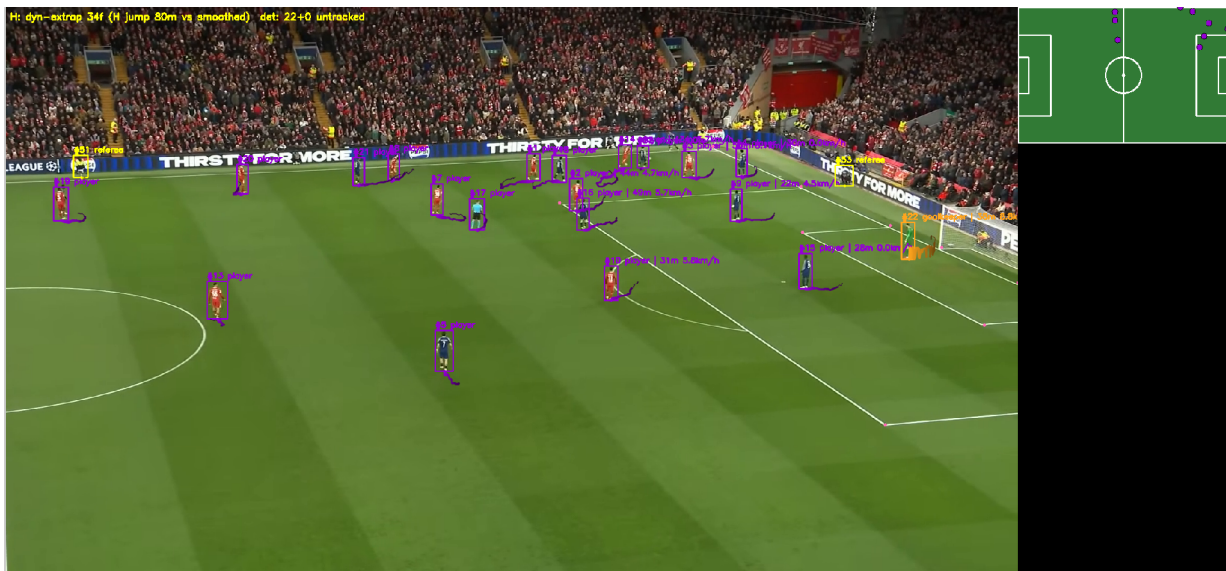
Rysunki 1–4 przedstawiają przykładowe klatki z działania programu na fragmencie meczu.



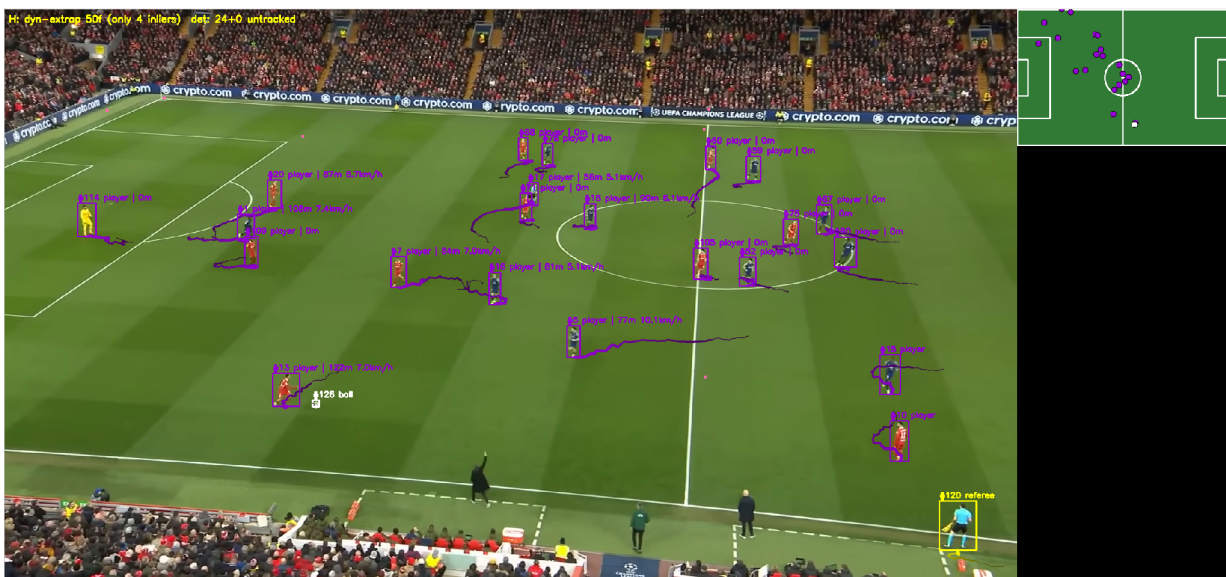
Rysunek 1: Ujęcie z dobrą homografią (H: dyn, 10 widocznych / 10 inlierów). Widoczne ramki zawodników z numerami ID, dystansem i prędkością, zanikające ślady trajektorii oraz minimapa 2D z naniesionymi pozycjami.



Rysunek 2: Inne ujęcie z poprawnie wyznaczoną homografią. Większość zawodników ma stabilne ID i wyliczone statystyki; bramkarz po prawej stronie jest poprawnie odróżniony (inny kolor ramki).



Rysunek 3: Ujęcie z ekstrakolowaną homografią (status `dyn-extrap`). Gdy model chwilowo gorzej widzi punkty boiska, utrzymywana jest ostatnia poprawna macierz; widać też wykrytego sędziego (oddzielny kolor ramki).



Rysunek 4: Szerokie ujęcie z widoczną piłką (`ball`) i sędzią (`referee`) – przykład klatki, w której rzadkie klasy zostały wykryte. Część etykiet prędkości pokazuje 0 km/h dla stojących zawodników.

5.2 Wyniki ilościowe (`process.py`)

Na potrzeby sprawozdania uruchomiono potok wsadowy na 20-sekundowym fragmencie (500 klatek, 1920×1080, 25 fps) jednego z klipów z repozytorium, przy użyciu wytrenowanych modeli `best.pt` i `pitch.pt` oraz karty GPU (CUDA). Tabela 1 zawiera podsumowanie wygenerowane przez `process.py`.

Tabela 1: Podsumowanie uruchomienia `process.py` (fragment 500 klatek).

Metryka	Wartość
Klatki wejściowe	500
Przepustowość	7,8 fps
Współczynnik czasu rzecz.	0,31×
Homografia zaufana	100% klatek
w tym świeże dopasowania	82%
w tym ekstrapolacja	18%
fallback (brak minimapy)	0%
Unikalne ID zawodników	39
Łączny dystans	878 m
Maks. prędkość końcowa	11,2 km/h

Z analizy zapisanego pliku `frames.jsonl` (rekord na każdą klatkę) wynika wyraźna nierównowaga w wykrywalności klas, co potwierdza obserwacje jakościowe. Tabela 2 pokazuje, w jakim odsetku klatek pojawiła się dana klasa oraz średnią liczbę śledzonych ramek na klatkę.

Tabela 2: Wykrywalność klas na fragmencie 500 klatek.

Klasa	Obecna w klatkach	Liczba detekcji (suma)
player (zawodnik)	500 / 500 (100,0%)	10 546
goalkeeper (bramkarz)	283 / 500 (56,6%)	286
referee (sędzia)	96 / 500 (19,2%)	108
ball (piłka)	55 / 500 (11,0%)	55

Średnio na klatkę przypadają 22 śledzone ramki (maksymalnie 25). Dane te jednoznacznie potwierdzają, że zawodnicy są wykrywani poprawnie (w każdej klatce), bramkarze w ponad połowie klatek, natomiast **sędziowie i piłka wykrywani są tylko sporadycznie** (odpowiednio ok. 19% i 11% klatek).

5.3 Stabilność homografii (`eval.py homography`)

Uruchomiono również ewaluację stabilności homografii na tym samym fragmencie (co trzecia klatka). Wyniki (Tabela 3) potwierdzają wysoką jakość dopasowań na ujęciach pokazujących wyraźny fragment boiska.

Tabela 3: Wynik `eval.py homography` (167 przetworzonych klatek).

Metryka	Wartość
Klatki z dopasowaną homografią	100,0%
Klatki z fallbackiem	0,0%
Widoczne punkty (śr. ± odch.)	8,7 ± 1,8
Widoczne punkty (min / max)	5 / 10
Inliery RANSAC (śr.)	7,8
Dryf macierzy klatka-do-klatki (L2)	0,77

Analiza widoczności poszczególnych punktów pokazała, że na badanym fragmencie (ujęcie prawej części boiska) punkty prawej strony – np. prawy górny róg (TR), narożniki prawego pola karnego i bramkowego – były widoczne w ok. 100% klatek, natomiast punkty lewej strony oraz środkowe (koło środkowe, punkty karne, wierzchołek łuku) w ok. 0% (są one albo poza kadrem, albo w

ogóle nieobecne w adnotacjach SoccerNet). To wyjaśnia, dlaczego pozycje na minimapie skupiają się po jednej stronie boiska, gdy kamera kadruje tylko jego fragment.

5.4 Co działa, a co nie

Działa dobrze:

- detekcja i śledzenie zawodników oraz bramkarzy (trwałe ID, ślady),
- wyznaczanie homografii i minimapa 2D na ujęciach z czytelnym boiskiem (100% dopasowań na testowym fragmencie),
- wyliczanie dystansu i prędkości (z realistycznymi wartościami i wygładzaniem),
- generowanie wideo z adnotacjami oraz raportów JSON.

Ograniczenia:

- piłka i sędziowie są wykrywani sporadycznie (mały/szybki obiekt oraz położenie na obrzeżach kadru),
- przy zbliżeniach na fragment boiska lub przy słabej widoczności linii homografia bywa ekstrapolowana, a minimapa rzadsza

Istotną przyczyną powyższych niedoskonałości - w szczególności jakości wykrywania punktów charakterystycznych boiska - były ograniczenia obliczeniowe po stronie zespołu. Trening modelu boiska jest kosztowny: na karcie NVIDIA RTX 3090 jedna epoka zajmowała około godziny, co przy konfiguracji liczącej 60-120 epok i zbiorze augmentowanym liczącym setki tysięcy obrazów realnie ogranicza liczbę eksperymentów, iteracji hiperparametrów oraz pełne przeprowadzenie dwuetapowego schematu trenowania (`train_pitch_partials.py` + `train_pitch_finetune.py`) opisanego w dokumentacji projektu. Przy większych zasobach obliczeniowych (szybsze GPU, większa pamięć VRAM, możliwość trenowania na pełnym zbiorze SoccerNet + Roboflow z augmentacją offline) wykrywanie keypointów boiska dałoby się znacznie poprawić – m.in. przez dłuższy trening, dokładniejsze dostrojenie na kadry częściowe boiska oraz wyższe metryki keypoint mAP, co bezpośrednio przekładałoby się na stabilniejszą homografię i wiarygodniejszą minimapę 2D.

6 Poradnik uruchomieniowy

6.1 Instalacja (conda + CUDA)

Projekt wymaga Pythona w wersji ≥ 3.10 , < 3.13 . Przykładowa instalacja w środowisku conda z obsługą GPU:

```
conda create -n fbtracker python=3.11 -y
conda activate fbtracker
pip install --upgrade pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
pip install -e ".[gpu,dev,training]"
# weryfikacja:
python -c "import torch; print(torch.cuda.is_available())"
python scripts/smoke_test.py
```

Kluczowe zależności (z `pyproject.toml`): `ultralytics` (YOLO26, trening i wnioskowanie), `supervision` (ByteTrack), `lap` (przypisania), `opencv-python`, `numpy`, `scipy`, `pandas`, `pyyaml`, `matplotlib`, `plotly`, `SoccerNet`, `roboflow`, `yt-dlp`, `rich`. Dodatki: `gpu` (`onnx`, `onnxruntime-gpu`), `dev` (`ruff`, `pytest`), `training` (`albumentations`). Na systemie Ubuntu przydatne bywają pakiety systemowe: `ffmpeg` `libgl1` `libglib2.0-0` `libsm6` `libxext6`.

6.2 Najważniejsza uwaga: kodek wideo

Przykładowe klipy w `samples/clips/` są zakodowane kodekiem **AV1**, którego wiele instalacji OpenCV nie potrafi zdekodować (objaw: `Could not read first frame`). Przed uruchomieniem warto przekonwertować materiał do H.264:

```
ffmpeg -i samples/clips/liv_psg_clip1.mp4 -t 20 -c:v libx264 -pix_fmt yuv420p -an clip_h264.mp4
```

6.3 Skrypty uruchomieniowe

demo.py – interaktywny podgląd całego potoku w oknie (wymaga środowiska graficznego). Otwiera wideo lub kamerę, na bieżąco wykrywa, śledzi, rzutuje na boisko i rysuje minimapę.

```
python demo.py --source clip_h264.mp4 --device cuda
python demo.py --source clip_h264.mp4 --output out.mp4 --no-show # tryb
    bezokienkowy
python demo.py --source clip_h264.mp4 --dump # zapis JSON
    na klatke
```

process.py – wsadowe, bezokienkowe przetwarzanie pliku wideo. Zawsze zapisuje wynikowy film oraz raporty JSON, a na końcu wypisuje tabelę podsumowującą. To zalecany sposób uzyskania powtarzalnych wyników.

```
python process.py clip_h264.mp4 -o output.mp4 --device cuda
python process.py clip_h264.mp4 --minimal # tylko ramki + ID + ślady
python process.py clip_h264.mp4 --no-minimap --no-analytics
```

eval.py – ewaluacja, czyli pomiar jakości:

- **eval.py detector** – liczy metryki detekcji (mAP, precyzja, czułość) detektora na zbiorze walidacyjnym; wymaga lokalnego zbioru detekcyjnego. Wynik: `outputs/reports/eval/detector-*.json`
- **eval.py pitch** – liczy metryki modelu boiska (mAP ramki i punktów); wymaga lokalnego zbioru boiska. Wynik: `outputs/reports/eval/pitch-*.json`.
- **eval.py homography** – sprawdza stabilność homografii na konkretnym filmie (odsetek dopasowań, powody odrzuceń, widoczność punktów); wymaga jedynie wideo i modelu boiska. Wynik: `outputs/reports/eval/homography-*.json`.

```
python eval.py homography --source clip_h264.mp4 --stride 3 --device cuda
python eval.py detector --weights models/checkpoints/best.pt --device cuda
python eval.py pitch --weights models/checkpoints/pitch.pt --device cuda
```

train.py / train_pitch_partials.py / train_pitch_finetune.py – trening modeli (opisany w sekcji 5). **dataset.py** – przygotowanie danych (sekcja 2). **scripts/smoke_test.py** – szybki test poprawności instalacji (importy modułów, render minimapy, schemat punktów, parsowanie `-help` skryptów); nie wymaga modeli, danych ani GPU.

6.4 Najszybsza weryfikacja działania

```
conda activate fbtracker
python scripts/smoke_test.py
ffmpeg -i samples/clips/liv_psg_clip1.mp4 -t 20 -c:v libx264 -pix_fmt yuv420p -an clip_h264.mp4
python process.py clip_h264.mp4 -o output.mp4 --device cuda
# wynik: output.mp4 + outputs/reports/demo/<nazwa>-<znacznik_czasu>/summary.json
```

7 Podsumowanie

W ramach projektu zrealizowano kompletny, dwumodelowy potok analizy nagrań piłkarskich: detektor YOLO26-s rozpoznaje zawodników, bramkarzy, sędziów i piłkę, tracker ByteTrack nadaje obiektom trwałe identyfikatory, a model YOLO26s-pose wyznacza 32 punkty charakterystyczne boiska, na podstawie których liczona jest homografia i budowana minimapa 2D z pozycjami zawodników. Dodatkowo system wylicza dystans i prędkość zawodników oraz zapisuje ustrukturyzowane raporty z każdego uruchomienia.

Przeprowadzone testy potwierdziły, że potok działa: na 20-sekundowym fragmencie homografia była wiarygodna w 100% klatek (82% świeżych dopasowań), wykryto 39 unikalnych identyfikatorów zawodników, a wartości dystansu i prędkości były realistyczne. Najmocniejszą stroną systemu jest detekcja i śledzenie zawodników oraz bramkarzy wraz z poprawnym odwzorowaniem ich pozycji na boisku. Głównymi ograniczeniami są: sporadyczna wykrywalność piłki i sędziów, zależność jakości minimapy od tego, jak wyraźnie kamera kadruje linie boiska, oraz ograniczone zasoby obliczeniowe przy trenowaniu modelu keypointów (ok. godzina na epokę na RTX 3090), co uniemożliwiło pełne wykorzystanie dwuetapowego schematu uczenia i ograniczyło dalszą poprawę detekcji punktów boiska.